

AI-Enhanced Image Processing and Management Web Application (HLD)

Group 2 Senior Project
Supervisor: Dr. Faisal Arafsha

Name	ID
Abdulrahman Alguydi	YB0062
Mohammed Dafterdar	VA0056
Saud Almansour	VA0040

Table of Contents

1. Introduction.....	3
1.1 Document Purpose	3
1.2 Project Scope	3
1.3 Definitions and Acronyms.....	3
2. System Architecture	4
2.1 Architectural Pattern	4
2.2 High-Level Architecture Flow.....	4
2.3 Component Description.....	5
3. Subsystem Decomposition.....	6
3.1 Overview	6
3.2 Client-Side Modules (Frontend).....	6
3.3 Server-Side Modules (Cloud Backend / AWS EC2).....	7
3.4 Data Storage & External Modules.....	8
4. Data Design.....	8
4.1 Overview	8
4.2 Entity-Relationship (ER) Model	8
4.3 Data Dictionary and Table Structures	9
4.4 Data Security and Integrity	10
5. Deployment & Environment	11
5.1 Overview	11
5.2 Deployment Architecture	11
5.3 Infrastructure Specifications.....	11
5.4 Version Control and Deployment Pipeline	12
6. System Sequence and Data Flow.....	12
6.1 Overview	12
6.2 Scenario 1: Image Upload and Validation.....	12
6.3 Scenario 2: Standard Image Modification (Local Processing).....	13
6.4 Scenario 3: AI-Powered Image Enhancement (OpenAI API).....	14
7. API and Interface Specifications.....	15
7.1 Overview	15
7.2 Authentication Endpoints	15
7.3 Upload and Gallery Endpoints	15
7.4 Processing Endpoints	16

1. Introduction

1.1 Document Purpose

The purpose of this High-Level Design (HLD) document is to define the overarching software architecture and structural blueprint for the AI-Enhanced Image Processing and Management Web Application. It translates the functional and non-functional requirements established in the Software Requirements Specification (SRS) into a concrete system design. This document will serve as the primary guideline for the development team, illustrating how the frontend, cloud backend, database, and external APIs interact to fulfill the project's objectives.

1.2 Project Scope

This document outlines the architectural framework of a full-stack, cloud-hosted image editing ecosystem. The scope covers the structural decomposition of the client-side user interface (built with HTML5, CSS3, and JavaScript), the cloud-based backend processing engine (driven by Python), and the persistent storage layer (managed MySQL). Standard geometric and color edits will be handled by the backend using libraries like OpenCV/Pillow, while all advanced AI features (upscaling and color correction) will be exclusively driven by secure integrations with the OpenAI API.

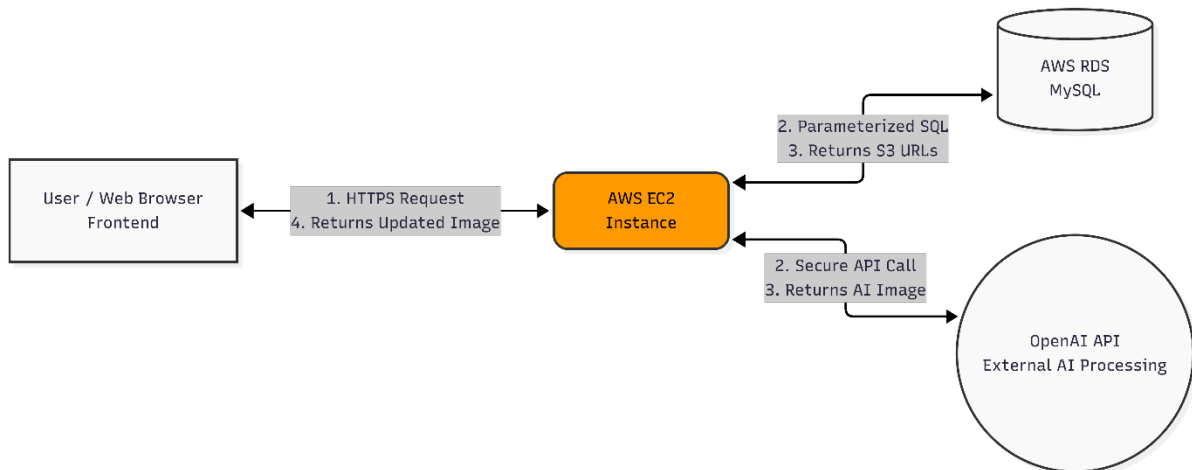
1.3 Definitions and Acronyms

- **API (Application Programming Interface):** A set of protocols used here to connect the backend server directly to OpenAI's infrastructure for image processing.
- **Cloud Hosting:** The delivery of computing resources (servers, storage, databases) over the internet, ensuring the application is accessible globally.
- **DBMS (Database Management System):** The software system used to manage and query the database (MySQL).
- **Client-Side:** The web browser environment where the user interacts with the application interface.
- **Server-Side:** The cloud-hosted backend infrastructure responsible for handling logic, routing, standard image edits, and API communication.

2. System Architecture

2.1 Architectural Pattern

The system adopts a **Cloud-Based Client-Server Architecture** utilizing a modular approach. The application is divided into three primary tiers: the Presentation Tier (Frontend), the Application Tier (Backend), and the Data Tier (Database). Because AI processing requires immense computational power, the system extends its architecture by utilizing an external Service-Oriented approach, offloading AI tasks to the OpenAI API.



2.2 High-Level Architecture Flow

The system processes data based on the type of edit requested by the user:

- Client Request:** The user uploads an image via the web browser, toggles their desired mode, and initiates an edit.
- Cloud Web Server Routing (Python):** The backend server (hosted on an AWS EC2 instance) receives the HTTPS request and evaluates the requested action:
 - Standard Edits:** If the user requests a standard manipulation (Crop, Flip, Swirl, Color Adjust), the server processes the image locally using its native Python libraries (e.g., OpenCV/Pillow).
 - AI Edits:** If the user requests AI Upscaling or AI Color Fix, the server acts as a secure proxy. It packages the image and sends it via RESTful API to OpenAI's servers. OpenAI processes the image and returns it to the AWS EC2 server.
- Data Persistence:** Once the image is processed, the Cloud Web Server connects to the Cloud Database (AWS RDS) to log the file metadata and save the asset to the user's personal gallery.
- Client Update:** The Cloud Web Server sends the final processed image back to the user's browser, triggering the Interactive Comparison Slider to update in real-time.

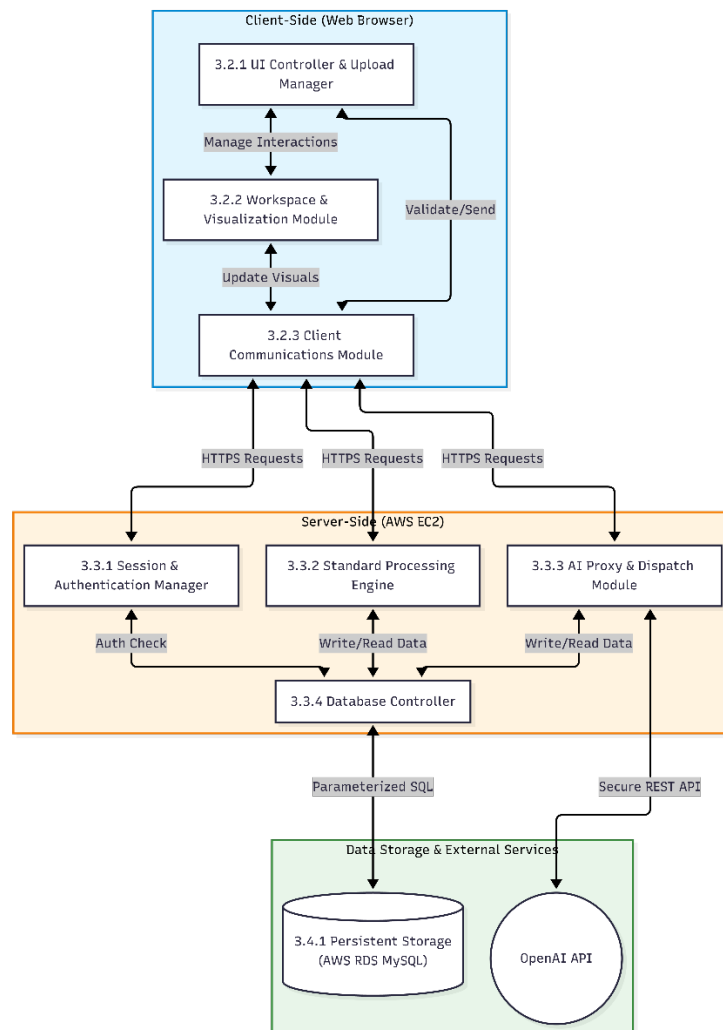
2.3 Component Description

- **Frontend (User Interface):** A dynamic web interface built with HTML, CSS, and JavaScript. It manages the user navigation between the upload hub and the personal gallery, and handles the interactive logic for the "Before vs. After" slider.
- **Cloud Backend Engine (AWS EC2):** A Python-based server hosted on an Amazon Elastic Compute Cloud (EC2) virtual machine. It is responsible for session management, enforcing the 1080p resolution limits, and executing standard geometric image processing locally.
- **OpenAI API Integration:** The external intelligence engine. The backend securely stores API keys as environment variables on the AWS EC2 server to ensure secure, authenticated communication for advanced AI tasks.
- **Cloud Database (AWS RDS for MySQL):** A managed Amazon Relational Database Service (RDS) instance storing user login credentials (hashed) and the file references for all original and modified images saved in user galleries.

3. Subsystem Decomposition

3.1 Overview

The system is decomposed into three primary operational domains: the Client-Side Application (Frontend), the Server-Side Engine (Backend), and the External Services (Database and API). Each domain consists of specialized modules responsible for fulfilling specific requirements outlined in the SRS.



3.2 Client-Side Modules (Frontend)

These modules run entirely within the user's web browser and manage all user interactions.

- **UI Controller & Upload Manager**
 - **Function:** Manages the web interface, drag-and-drop inputs, and initial file validation before sending data to the server. It also handles reloading images from the gallery.
 - **Traceability:** Fulfills **REQ 3.1-1** (Validate JPG/PNG formats), **REQ 3.1-2** (Block uploads exceeding 1080p), **REQ 3.1-5** (Display error messages and clear session on failure), and **REQ 3.5-5** (Reload gallery image into workspace).

- **Workspace & Visualization Module**

- **Function:** Handles the rendering of the image workspace. It is explicitly responsible for managing the logic of the "Before vs. After" visual split and centering images.
- **Traceability:** Fulfills **REQ 3.4-1** (Display interactive slider), **REQ 3.4-2** (Show before/after separated by slider), **REQ 3.4-3** (Update visual split in real-time), **REQ 3.4-4** (Ensure slider works for all modifications), and **REQ 3.4-5** (Center images if aspect ratio changes).

- **Client Communications Module**

- **Function:** Packages user inputs and securely transmits them to the backend server.
- **Traceability:** Fulfills **REQ 5.2-6** (Utilize HTTPS for all data transmissions).

3.3 Server-Side Modules (Cloud Backend / AWS EC2)

These modules reside on the Python-based cloud server and act as the core logic engine.

- **Session & Authentication Manager**

- **Function:** Manages login states and determines if a user is a Guest (temporary memory) or a Registered User (database access).
- **Traceability:** Fulfills **REQ 5.2-1** (Require unique username and password), **REQ 3.1-4** (Guest user images kept in temporary session), and **REQ 5.2-4** (Ensure Guest user images are not saved server-side).

- **Standard Processing Engine**

- **Function:** Utilizes native Python libraries to execute local image modifications.
- **Traceability:** Fulfills **REQ 3.2-1** (Apply geometric transformations like crop/flip/swirl), **REQ 3.2-2** (Adjust colors and brightness), **REQ 3.2-3** (Ensure output quality is equal to original), and **REQ 3.2-5** (Display modified output for Guests without DB records).

- **AI Proxy & Dispatch Module**

- **Function:** Acts as the secure gateway to external AI. It receives AI requests and transmits the image to OpenAI for processing.
- **Traceability:** Fulfills **REQ 3.3-1** (Apply AI upscaling), **REQ 3.3-2** (Apply automated color correction), **REQ 3.3-4** (Ensure AI output quality $\geq$ original), and **REQ 5.2-8** (Secure API keys stored as environment variables).

- **Database Controller**

- **Function:** Acts as the exclusive bridge between the Python server and the AWS RDS MySQL database. It dictates *when* data is saved and sanitizes all incoming SQL queries.
- **Traceability:** Fulfills **REQ 3.1-3** (Save to DB only *after* an edit tool is applied), **REQ 3.2-4** (Save original & modified images for standard presets), **REQ 3.3-3** (Save original & modified images for AI presets), and **REQ 5.2-7** (Use parameterized queries to prevent SQL injection).

3.4 Data Storage & External Modules

- **Persistent Storage Module (AWS RDS MySQL)**
 - **Function:** The relational database that permanently stores user credentials and references to user-owned images. It enforces strict data isolation and privacy rules.
 - **Traceability:** Fulfills **REQ 5.2-2** (Store passwords using cryptographic hashes), **REQ 3.5-1** (Retrieve images linked to specific user), **REQ 3.5-2** (Allow deletion of a specific image), **REQ 3.5-3** (Permanently remove the image file and associated data upon deletion), **REQ 3.5-4** (Ensure users only view their own images), **REQ 5.2-3** (Access/deletion restricted to original uploader), and **REQ 5.2-5** (Do not share images in a public gallery).

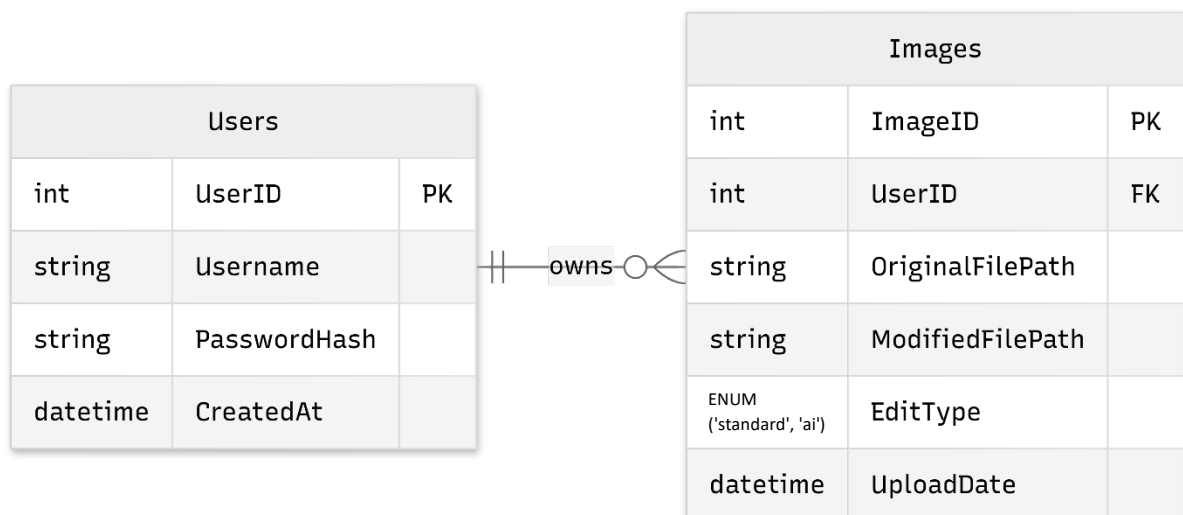
4. Data Design

4.1 Overview

The data design for the application relies on a relational database management system, specifically MySQL hosted on AWS RDS. The database is strictly designed to manage Registered Users. Guest User data is handled entirely in temporary session memory and does not interact with this data layer, fulfilling **REQ 5.2-4**.

4.2 Entity-Relationship (ER) Model

The database utilizes a simple but secure relational model consisting of two primary entities: "Users" and "Images". A one-to-many (1:N) relationship exists between the User and the Images, meaning one user can own multiple images, but each image record belongs strictly to one user.



4.3 Data Dictionary and Table Structures

The database is normalized to ensure data integrity and fast retrieval times. Below are the definitions for the core tables.

The "Users" Table

This table stores the authentication credentials for Registered Users.

Users		
int	UserID	PK
string	Username	
string	PasswordHash	
datetime	CreatedAt	

- **UserID (Primary Key):** An auto-incrementing integer serving as the unique identifier for each account.
- **Username (VARCHAR):** The unique login name chosen by the user.
- **PasswordHash (VARCHAR):** The user's password, stored strictly as a one-way cryptographic hash to ensure plain-text passwords are never accessible.
 - *Traceability:* Fulfills **REQ 5.2-1** and **REQ 5.2-2**.
- **CreatedAt (DATETIME):** A timestamp recording when the account was created.

The "Images" Table

This table acts as a ledger, storing the metadata and secure cloud file paths (e.g., AWS S3 bucket links) for the user's gallery. It does *not* store the actual image files directly in the rows, ensuring high performance.

Images		
int	ImageID	PK
int	UserID	FK
string	OriginalFilePath	
string	ModifiedFilePath	
ENUM	EditType	
datetime	UploadDate	

- **ImageID (Primary Key):** An auto-incrementing integer serving as the unique identifier for each image session.
- **UserID (Foreign Key):** Links the image directly to the specific user who uploaded it, ensuring strict access control.
 - *Traceability:* Fulfills **REQ 3.5-4** (Users only view their own images) and **REQ 5.2-3** (Access restricted to original uploader).
- **OriginalFilePath (VARCHAR):** The secure cloud storage link to the original, untouched 1080p image.

- **ModifiedFilePath (VARCHAR):** The secure cloud storage link to the final image after standard or AI edits have been applied.
 - *Traceability:* Fulfills **REQ 3.1-3**, **REQ 3.2-4**, and **REQ 3.3-3** (Saving both original and modified images to the database upon editing).
- **EditType (ENUM('standard', 'ai')):** Restricts input values strictly to 'standard' or 'ai' to enforce database-layer integrity and classify the manipulation.
- **UploadDate (DATETIME):** A timestamp recording when the file was saved.

4.4 Data Security and Integrity

To maintain the security of the database and prevent unauthorized data manipulation:

- **Data Isolation:** The database query logic enforces that any "SELECT", "UPDATE", or "DELETE" command must include a "WHERE UserID = 'ActiveUser'" clause to prevent cross-account access.
 - *Traceability:* Fulfills **REQ 3.5-1**, **REQ 3.5-2**, and **REQ 3.5-3**.
- **SQL Injection Prevention:** The backend Database Controller module forces all inputs through parameterized queries before they reach the MySQL instance.
 - *Traceability:* Fulfills **REQ 5.2-7**.

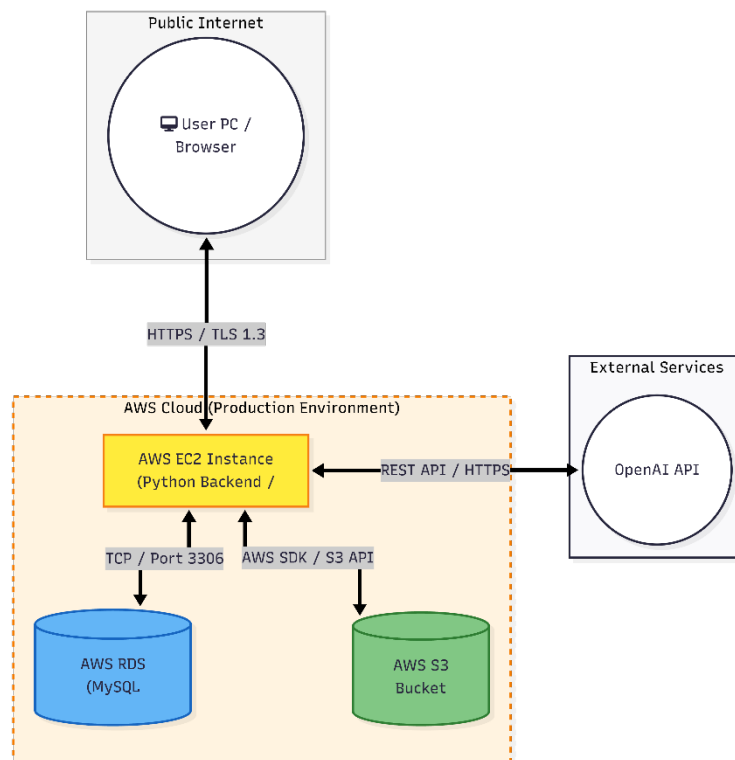
5. Deployment & Environment

5.1 Overview

The deployment strategy utilizes a cloud-native approach hosted entirely on **Amazon Web Services (AWS)**. This ensures high availability, scalability, and secure data management. The system is separated into distinct development and production environments to ensure that new code is tested before it reaches the end users.

5.2 Deployment Architecture

The production environment consists of interconnected AWS services that map directly to the software subsystems defined in Section 3.



5.3 Infrastructure Specifications

- **Web Server (AWS EC2):** The Python backend (Flask/FastAPI) and the compiled frontend UI will be hosted on an EC2 instance. This instance acts as the central hub, processing all incoming user requests.
- **Database Server (AWS RDS):** A managed MySQL instance will run on RDS. It is placed within a Virtual Private Cloud (VPC) so that it is only accessible by the EC2 web server, preventing direct outside access from the internet.
- **File Storage (AWS S3):** Amazon Simple Storage Service (S3) will be used as a secure bucket to store the physical .jpg and .png files. The database will only store the secure URLs to these S3 files (the OriginalFilePath and ModifiedFilePath).

- **Security & SSL:** The domain will be secured using an SSL/TLS certificate (e.g., via AWS Certificate Manager) to ensure all traffic between the user's browser and the EC2 server is encrypted.
 - *Traceability:* Fulfills **REQ 5.2-6** (Utilize HTTPS for all data transmissions).

5.4 Version Control and Deployment Pipeline

To maintain code quality and track bugs efficiently during the development lifecycle, the team will utilize modern version control and issue tracking:

- **Issue Tracking:** The team will exclusively use **GitHub Issues** to track bugs, plan features, and manage sprint tasks.
- **Version Control:** All source code will be hosted on GitHub.
- **Deployment:** When code is marked as stable in the main GitHub repository, it will be pulled onto the AWS EC2 instance for production deployment.

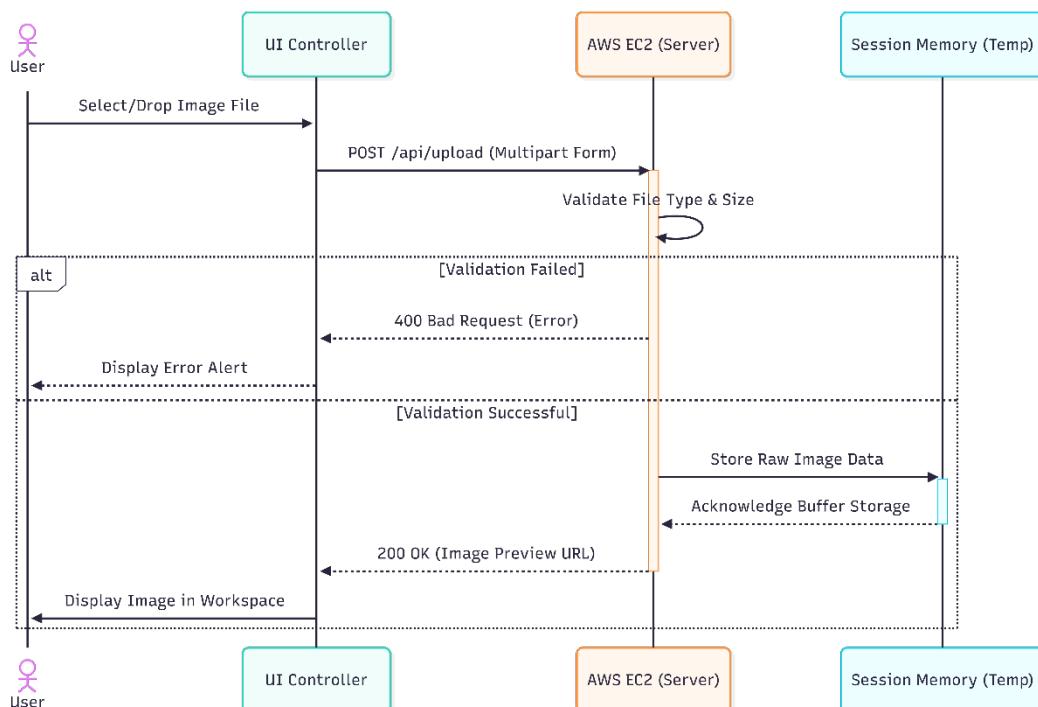
6. System Sequence and Data Flow

6.1 Overview

While the previous sections defined the structural components of the system, this section details the dynamic interactions between those components. The data flow is illustrated using Unified Modeling Language (UML) Sequence Diagrams to represent the chronological step-by-step execution of core system features, ensuring all data hops are secure and efficient.

6.2 Scenario 1: Image Upload and Validation

This sequence describes the system's behavior when a user first uploads an image, demonstrating how the system handles the memory temporarily before any edits are made.

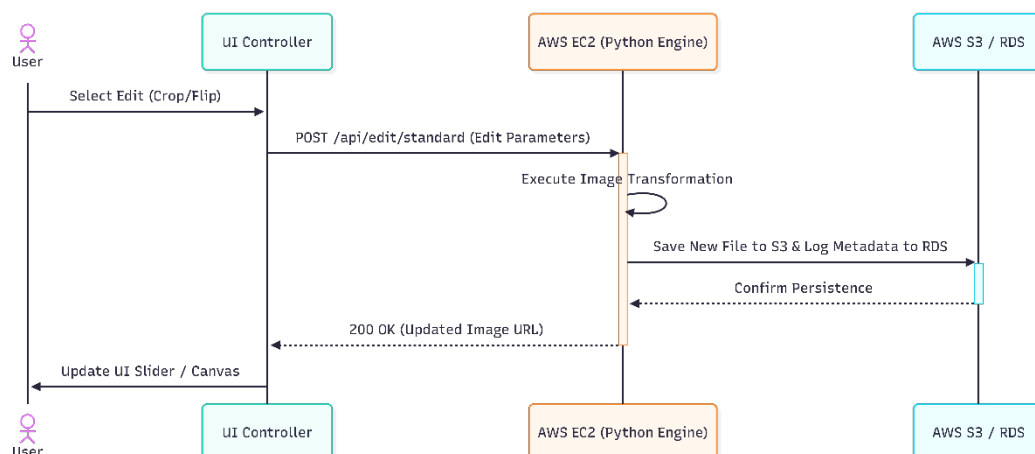


Step-by-Step Data Flow:

1. **Action:** The user drags and drops an image into the Central Upload Hub.
 2. **Transmission:** The UI Controller transmits the file via HTTPS to the AWS EC2 Web Server.
 3. **Validation:** The EC2 Server's logic engine verifies the file format (JPG/PNG) and checks that the resolution does not exceed 1080p.
 4. **Temporary Storage:** Upon successful validation, the EC2 Server holds the image in active session memory. *Note: The database (AWS RDS) and permanent storage (AWS S3) are not contacted during this initial upload phase.*
 5. **UI Update:** The server returns a success token, and the image is rendered in the user's workspace.
- **Traceability:** Fulfills **REQ 3.1-1** (Format validation), **REQ 3.1-2** (1080p limit), and **REQ 3.1-4 / REQ 5.2-4** (Guest user session memory limits).

6.3 Scenario 2: Standard Image Modification (Local Processing)

This sequence details how a standard, non-AI edit (like Crop or Flip) is processed entirely within our AWS infrastructure.



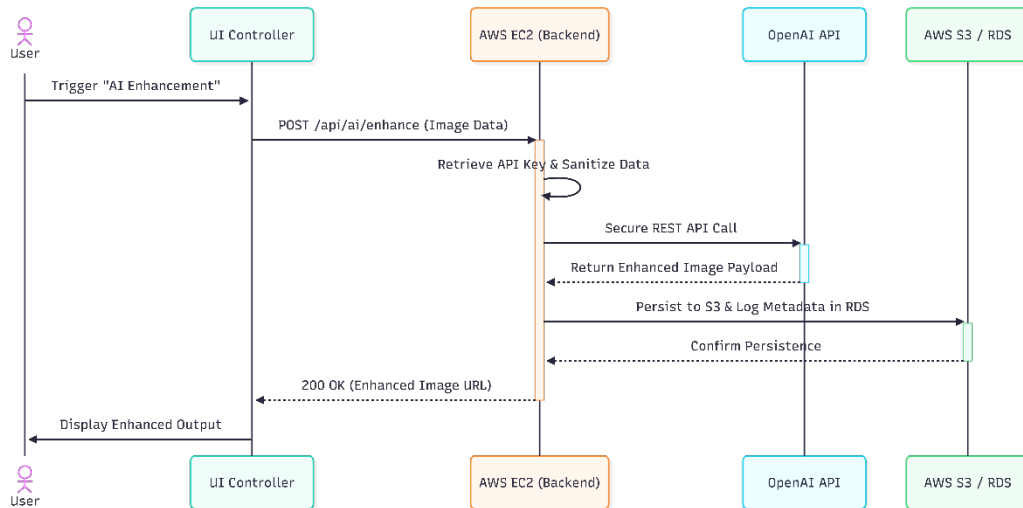
Step-by-Step Data Flow:

1. **Action:** A Registered User selects the "Flip" tool and clicks apply.
2. **Transmission:** The UI sends a JSON request detailing the edit parameters to the AWS EC2 Server.
3. **Processing:** The EC2 Server utilizes its local Python libraries (e.g., OpenCV) to execute the geometric transformation on the image held in session memory.
4. **Storage (AWS S3):** The EC2 Server uploads both the original 1080p image and the newly modified image to the secure AWS S3 bucket. S3 returns two specific file URLs.
5. **Logging (AWS RDS):** The EC2 Server executes a parameterized SQL INSERT query, saving the user's ID, the edit type, and the two S3 URLs into the Images table.
6. **UI Update:** The EC2 server sends the URLs back to the frontend, instantly activating the Interactive Comparison Slider.

- **Traceability:** Fulfills **REQ 3.2-1** (Geometric transformations), **REQ 3.2-4** (Save original & modified images), and **REQ 3.4-3** (Update visual split).

6.4 Scenario 3: AI-Powered Image Enhancement (OpenAI API)

This sequence maps out the most complex action in the system: securely offloading heavy processing to the external OpenAI API.



Step-by-Step Data Flow:

1. **Action:** A Registered User selects the "AI Upscale" tool.
 2. **Trigger:** The UI displays a processing spinner to prevent duplicate requests and sends the command to the AWS EC2 Server.
 3. **API Dispatch:** The EC2 Server acts as a secure proxy. It retrieves its hidden OpenAI API keys from its environment variables, packages the 1080p image data, and initiates a secure RESTful POST request to OpenAI's servers.
 4. **External Processing:** OpenAI processes the image and returns the enhanced high-quality asset back to the EC2 Server.
 5. **Persistence:** Just like Scenario 2, the EC2 Server saves the files to AWS S3 and logs the URLs in the AWS RDS MySQL database.
 6. **UI Update:** The UI removes the loading spinner and updates the Interactive Comparison Slider with the AI-enhanced result.
- **Traceability:** Fulfills **REQ 3.3-1** (AI upscaling), **REQ 3.3-3** (Save AI outputs to DB), and **REQ 5.2-8** (Secure API key handling).

7. API and Interface Specifications

7.1 Overview

This section defines the RESTful Application Programming Interface (API) that dictates how the Client-Side (Frontend) communicates with the Server-Side Engine (AWS EC2). All communications will occur over HTTPS, utilizing JSON payloads for data transfer to ensure a lightweight and secure connection.

7.2 Authentication Endpoints

These endpoints manage user access and session states.

- **POST /api/auth/register**
 - **Description:** Creates a new Registered User account.
 - **Payload:** {"username": "text", "password": "text"}
 - **Server Action:** Hashes the password and saves the credentials to the AWS RDS database.
 - **Traceability:** Fulfills **REQ 5.2-1** and **REQ 5.2-2**.
- **POST /api/auth/login**
 - **Description:** Authenticates an existing user and initiates a secure session.
 - **Payload:** {"username": "text", "password": "text"}
 - **Server Action:** Verifies the password hash and returns a secure session token (e.g., JWT) to the browser.

7.3 Upload and Gallery Endpoints

These endpoints handle the transfer of image files and the management of the persistent user library.

- **POST /api/upload**
 - **Description:** Handles the initial file upload from the Central Upload Hub.
 - **Payload:** Multipart/form-data containing the image file.
 - **Server Action:** Validates that the file is a JPG or PNG and does not exceed 1080p. If valid, the server assigns a temporary session ID and holds the image in memory.
 - **Traceability:** Fulfills **REQ 3.1-1**, **REQ 3.1-2**, and **REQ 3.1-4**.
- **GET /api/gallery**
 - **Description:** Retrieves the list of saved images for the active user.
 - **Server Action:** Queries the AWS RDS database (WHERE UserID = 'ActiveUser') and returns a JSON array containing the metadata and AWS S3 URLs for both original and modified images.
 - **Traceability:** Fulfills **REQ 3.5-1** and **REQ 3.5-4**.

- **DELETE /api/gallery/{ImageID}**
 - **Description:** Permanently deletes a specific image record.
 - **Server Action:** Verifies the user owns the image, deletes the row from the AWS RDS database, and commands AWS S3 to delete the physical files.
 - **Traceability:** Fulfills **REQ 3.5-2**, **REQ 3.5-3**, and **REQ 5.2-3**.

7.4 Processing Endpoints

These endpoints trigger the actual image modifications.

- **POST /api/edit/standard**
 - **Description:** Executes local geometric and color modifications.
 - **Payload:** {"tool": "crop|flip|swirl", "parameters": {...}}
 - **Server Action:** Applies the edit using Python libraries. If the user is registered, it saves the result to AWS S3 and logs it in AWS RDS. It returns the final image URL to the frontend.
 - **Traceability:** Fulfills **REQ 3.2-1**, **REQ 3.2-2**, and **REQ 3.2-4**.
- **POST /api/edit/ai**
 - **Description:** Triggers advanced AI enhancements.
 - **Payload:** {"tool": "upscale|color_fix"}
 - **Server Action:** Acts as a secure proxy, attaching the hidden API key, and forwarding the image to the OpenAI API. Upon receiving the result, it saves the files to AWS S3, logs them in AWS RDS (for registered users), and returns the final URL.
 - **Traceability:** Fulfills **REQ 3.3-1**, **REQ 3.3-2**, and **REQ 5.2-8**.